

WorldEpisode: An Executable Interchange Contract Linking Robot-Learning Episodes to Versioned, Replayable 3D Worlds

Alba Tellez Fernandez
URDF Studio
WorldEpisode draft maintainer

WorldEpisode Contributors

Abstract

Robot-learning datasets standardize observation–action trajectories, while 3D scene formats standardize visual and physical assets. However, the correspondence between an episode and the world in which it occurred remains encoded through dataset-specific assumptions, preventing reliable reconstruction, replay, conversion, counterfactual augmentation, and leakage-safe evaluation. We introduce *WorldEpisode*, a storage- and representation-neutral interchange profile that binds robot-learning episodes to immutable world revisions through explicit clocks and coordinate frames, persistent entity identity, fully specified action semantics, multi-representation asset roles, interaction events, provenance, and uncertainty. WorldEpisode does not replace existing containers or scene formats; it provides bindings for LeRobotDataset, Rerun, NCore, MCAP, OpenUSD, glTF Gaussian splats, and simulator-specific world targets. We implement a draft validator and deterministic pilot suite that measures semantic retention across seven bindings, injects fourteen physical-coherence faults, compares declared-latency replay against naive command-time replay, tests lineage-safe split leakage, and evaluates a synthetic counterfactual-augmentation task. The pilot suite detects all injected fault classes, improves replay RMSE by $3.6\times$ over naive timing, and exposes perfect leakage in random episode splits. These results do not replace large real-robot evaluation, but they establish that the proposed contract is executable and that its missing semantics affect measurable outcomes.

1 Introduction

Robot-learning datasets and 3D scene formats are improving quickly, but they solve different problems. LeRobot-style datasets focus on efficient observation–action episodes. Rerun provides a physical-data layer and storage target for multimodal recordings. NCore documents explicit pose graphs and sensor conventions. OpenUSD, SimReady, glTF, and simulator-native formats describe composed worlds and assets. Gaussian splats are also moving into standard asset containers through glTF’s `KHR_gaussian_splatting` extension [5, 6]. Recent real-to-sim systems such as RoboSnap and DROID-Sim show that Gaussian-splat environments can support trajectory replay, synthetic data generation, and policy evaluation at dataset scale [4].

This landscape rules out a weak framing: WorldEpisode is not the first format that connects robot data and Gaussian-splat worlds. That space is already occupied by asset standards and method papers. The remaining opening is the contract connecting all of them: the persistent, auditable binding between a robot-learning episode and the versioned world in which it occurred.

WorldEpisode targets that missing semantic layer. It is:

- **storage-neutral**: serializable through LeRobot, Rerun, NCore, MCAP, or a reference package;

- **representation-neutral:** compatible with splats, meshes, NeRFs, point clouds, and future assets;
- **runtime-neutral:** transferable to Isaac Sim, MuJoCo, SAPIEN, Genesis, and other simulators;
- **loss-explicit:** conversions may be lossy, but never silently lossy.

The paper’s core claim is therefore not a new monolithic file format. It is an executable interchange profile for world–episode semantics: identity, frames, clocks, action contracts, representation roles, events, provenance, uncertainty, and lineage-safe evaluation splits. The current artifact includes a JSON Schema, semantic validator, conversion-retention model, and deterministic pilot experiments. The experiments are deliberately small; their role is to test whether the proposed semantics are machine-checkable and empirically consequential before scaling to real-robot benchmarks.

2 Problem Statement and Scope

WorldEpisode binds an episode to a world revision. The base state is:

$$W_e(t) = W^{(r)} \oplus \Delta W_e(t), \quad (1)$$

where $W^{(r)}$ is an immutable, content-addressed world revision, $\Delta W_e(t)$ is the ordered sequence of episode-specific state changes, e is a robot-learning episode, and t is expressed in an explicitly declared clock domain.

An episode is modeled as:

$$E = \langle M, W^{(r)}, B, K, F, C, O, A, X, V, P, Q \rangle, \quad (2)$$

where M is the manifest, B the embodiment, K task/outcome semantics, F the frame graph, C clocks, O persistent entities and representations, A action-space contract, X synchronized trace, V world deltas and events, P provenance, and Q quality and uncertainty records.

Version 1 is intentionally narrow: rigid tabletop manipulation with fixed-base single- or dual-arm robots. Humanoids, locomotion, deformables, fluids, and multi-agent worlds are later profiles. This keeps conformance executable rather than aspirational.

WorldEpisode reuses existing standards rather than replacing them. USD [1] carries composed simulation worlds, glTF carries Gaussian appearance assets [5], NCore carries capture and pose-graph conventions [2], Rerun carries physical-data recordings with column chunks [3], and ROS/MCAP carries logs. WorldEpisode specifies the semantic relationships those bindings must preserve.

3 The Five-Graph Model

3.1 Identity Graph

Every physical or logical entity receives a persistent identifier: robot, link, gripper, sensor, rigid object, articulated object, fixture, support surface, region, human, light, or semantic group. The same `entity_id` must be used by segmentation masks, bounding boxes, Gaussian groups, render meshes, collision bodies, simulator actors, language references, contact events, object trajectories, grasp events, and attachment events.

3.2 Frame and Clock Graph

Every spatial value declares source frame, target frame, units, handedness, up axis, transform direction, quaternion convention, validity interval, calibration revision, and uncertainty where available. Every stream declares a clock domain, device timestamp, host-receive timestamp where available, exposure interval for cameras, effective command time for actions, synchronization mapping, and estimated offset/drift/error. NCore’s explicit pose-graph convention demonstrates the kind of rigor WorldEpisode requires across the whole episode [2].

3.3 Representation-Role Graph

One physical entity can have many representations: Gaussian splat appearance, textured render mesh, metric surface mesh, depth point cloud, collision proxy, mass/inertia record, semantic affordance, or learned embedding. Each representation declares its role and an asset descriptor: URI, media type, SHA-256 digest, optional mirrors, optional embedded payload metadata, coordinate frame, units, validity interval, derivation, uncertainty, and license. The URI may be relative, HTTP(S), Hugging Face, object storage, OCI, IPFS, or another registered resolver. Portability comes from deterministic resolution and digest verification, not from requiring every asset to live in one folder. The role field is essential: a splat may be valid for appearance but invalid for collision.

3.4 Temporal State and Event Graph

Episodes do not store only poses. They record interaction events: contact begin/end, grasp establish/release, attachment create/remove, support, containment, articulation state changes, spawn/despawn, topology changes, simulator reset, intervention, subgoal completion, and failure detection. Each event references entities, time, confidence, source, and optionally evidence.

3.5 Provenance and Derivation Graph

Every derived asset identifies source episodes, source sensors and intervals, reconstruction algorithm, software version, model/checkpoint hash, configuration, calibration revision, manual edits, source and output hashes, creator, license, and quality metrics. WorldEpisode maps to existing provenance and dataset metadata vocabularies rather than inventing an incompatible publication vocabulary.

4 Action Semantics, Revisions, and Loss Reports

4.1 Complete Action Semantics

An action vector is not portable without its operational contract. Each action channel must declare actuator, control mode, parameterization, reference frame, units, absolute/delta/velocity semantics, limits, command rate, command timestamp, effective timestamp, latency model, interpolation, normalization, gripper semantics, and missing-value policy. A vector such as $[0.01, 0, 0, 0, 0, 0, 1]$ is otherwise ambiguous: it could be a tool-frame delta, base-frame delta, velocity command, normalized policy output, or delayed target increment.

4.2 Immutable World Revisions

Each episode references one base world revision plus ordered episode deltas. A new revision is required when world geometry, metric scale, calibration, collision proxies, physical parameters, entity

decomposition, object identity, or task-relevant fixtures change. Content hashes allow two datasets to prove that they refer to the same world without relying on filenames.

4.3 Loss-Aware Conversion

WorldEpisode supports many bindings, but it does not promise impossible universal losslessness. Every converter emits a machine-readable report listing preserved, externalized, approximated, discarded, and warning fields. The standard permits lossy conversion; it forbids silent lossy conversion.

4.4 Implementation Artifact

The repository contains a JSON Schema, a machine-readable conformance requirement file, a semantic validator, and a deterministic experiment runner. The validator emits stable requirement identifiers such as `TIME.001`, `FRAME.002`, and `ASSET.002`. The experiment runner uses the minimal WorldEpisode package as a baseline, injects controlled faults, evaluates declared binding capabilities, and writes reproducible result artifacts. URDF Studio remains the initial implementation root, but the contract is represented independently in the public schema and validator.

5 Pilot Experiments

We evaluate the draft contract with deterministic pilot experiments in the public repository. The suite is not a replacement for real-robot benchmarking; it tests whether the proposed semantics are machine-checkable and whether omitting them changes measurable outcomes.

RQ1: semantic preservation across bindings. We define 23 semantic fields and score how much each target binding preserves natively versus with a WorldEpisode sidecar. Table 1 shows that common containers preserve only 17–39% of the contract natively, while the sidecar recovers all fields for dataset/log/world bindings. A Gaussian asset binding alone preserves only 17% natively and 30% with limited sidecar metadata, confirming that an asset format is not an episode–world contract.

Table 1: Semantic field retention in the pilot binding model.

Binding	Native	With WorldEpisode
LeRobot v3	0.261	1.000
Rerun RRD	0.348	1.000
NCore	0.304	1.000
MCAP/ROS2	0.261	1.000
OpenUSD/SimReady	0.391	1.000
glTF Gaussian asset	0.174	0.304

RQ2: physical-coherence fault detection. We inject 14 faults into the baseline package, including missing clock domains, incomplete cross-clock mappings, unknown frames, missing transform intervals, unknown event entities, missing representation roles, digest errors, action-contract omissions, non-content-addressed worlds, missing delta sequences, missing provenance, and missing quality records. The validator detects all 14 expected requirement failures with 1.000 recall and

0.933 precision. The single false-positive requirement is useful rather than harmful: removing an action reference frame triggers both `ACTION.001` and the downstream `FRAME.001` reference check.

RQ3: declared replay timing. We replay a deterministic grasp-and-transport trace with a two-step declared actuation latency. The declared-latency replay obtains object-trajectory RMSE 0.00395, while naive command-time replay obtains RMSE 0.01422, a $3.6\times$ improvement. A second engine parameterization under the same declared assumptions obtains RMSE 0.00668, illustrating replay within a tolerance envelope rather than bit-identical physics.

RQ4: synthetic counterfactual robustness. We evaluate a shifted-camera/object-displacement regression task. Observations-only nearest-neighbor control succeeds on 0.292 of shifted test cases. Adding unstructured 3D side information increases success to 0.342. WorldEpisode-style world-frame identity and counterfactual camera augmentation increase success to 0.592 over 120 deterministic test cases.

RQ5: lineage-safe splits. We generate episodes with world/entity lineage labels and evaluate a memorization policy. A random episode split leaks every tested world/entity pair and obtains 1.000 accuracy. World-disjoint and entity-disjoint splits have zero measured leakage and reduce memorization accuracy to 0.500 and 0.562 respectively. This pilot demonstrates why split manifests need lineage constraints.

6 Discussion

Not another monolithic binary. WorldEpisode has a small semantic core and multiple bindings. LeRobot is the policy-training view; Rerun is the multirate physical-data view; NCore is the sensor/reconstruction capture view; MCAP/ROS is the raw log view; OpenUSD/SimReady is the composed simulation-world view; glTF or SPZ is the Gaussian asset view; and a reference directory is the conformance profile.

Lineage-safe evaluation. Episode-level random splits can leak the same reconstructed room, physical object, Gaussian asset, source video, world revision, or generated asset lineage across training and evaluation. WorldEpisode supports split constraints such as `world_lineage`, `physical_entity`, `source_capture`, `reconstruction_run`, and `generated_asset_family`.

Standards alignment. The project aligns with standards bodies rather than antagonizing them. Static 3D robot-map exchange, humanoid dataset lifecycle, OpenLABEL annotation, glTF Gaussian splats, USD composition, and NCore capture conventions are all useful neighbors. WorldEpisode is an executable profile and bridge across those layers.

Limitations. The current experiments are deterministic pilots, not a full robotics benchmark. They do not train a VLA, run on a real robot, or prove cross-lab adoption. Their narrower contribution is to show that the contract is implementable, that requirement failures can be detected mechanically, and that timing, lineage, and world-frame identity affect measured outcomes. The next empirical step is to run the same protocol on real multi-camera manipulation episodes and independent adapters.

7 Conclusion

WorldEpisode reframes the opportunity away from “LeRobot plus Gaussian splats in one file” and toward persistent, verifiable world-episode interoperability. Gaussian splats remain a high-value profile, but the durable contribution is a storage-neutral contract for immutable world revisions, persistent entity identity, frame and clock graphs, action semantics, representation roles, events, provenance, uncertainty, loss-aware conversion, and lineage-safe evaluation. The pilot implementation turns those requirements into executable checks and reports measurable effects for conversion loss, fault detection, replay timing, counterfactual augmentation, and leakage-safe splitting. The next step is to replace the synthetic pilots with real multi-robot datasets and independently maintained bindings.

References

- [1] Alliance for OpenUSD. OpenUSD: Universal scene description. <https://openusd.org/>, 2026. Accessed 2026-07-13.
- [2] NVIDIA. NCore Specification: Data Conventions. <https://nvidia.github.io/ncore/data/conventions>, 2026. Accessed 2026-07-13.
- [3] Rerun. A new data layer for robot learning. <https://rerun.io/blog/data-layer-for-robot-learning>, 2026. Accessed 2026-07-13.
- [4] RoboSnap Authors. RoboSnap: One-Shot Real-to-Sim Scene Generation for Generalizable Robot Learning and Evaluation. <https://arxiv.org/html/2607.06699v1>, 2026. Accessed 2026-07-13.
- [5] The Khronos Group. KHR_gaussian_splatting. https://github.com/KhronosGroup/glTF/blob/main/extensions/2.0/Khronos/KHR_gaussian_splatting/README.md, 2026. Accessed 2026-07-13.
- [6] The Khronos Group. Khronos Announces glTF Gaussian Splatting Extension. <https://www.khronos.org/news/press/glTF-gaussian-splatting-press-release>, 2026. Accessed 2026-07-13.